

Bloque 01_fundamentos

Índice de módulos

- Bases de datos y flujo de datos
- Encoding, normalización y parsers
- HTTP, formularios, cookies y sesiones
- Linux, procesos, permisos y shell
- Redes, DNS, puertos y URLs

Bases de datos y flujo de datos

Objetivos

Explicar cómo se construye una consulta, modelar source-transformación-validación-sink y distinguir datos de estructura.

Consulta y parámetros

Una base relacional recibe una sentencia que el motor tokeniza y analiza. Concatenar entrada dentro de esa sentencia permite que los caracteres del usuario cambien su estructura. Una consulta parametrizada envía estructura y valores por canales lógicos separados; el valor no se vuelve sintaxis SQL.

```
Inseguro: texto SQL + entrada -> parser SQL
Seguro:  sentencia con marcador -> parser/preparación
        valor separado          -> binding tipado
```

Parametrizar valores no resuelve por sí solo identificadores dinámicos como nombre de columna u orden. Para esos casos se selecciona entre valores permitidos definidos por la aplicación.

Modelo de flujo

- Source: punto controlable, por ejemplo parámetro HTTP, cookie, fichero o variable de entorno.
- Transformación: decodificación, concatenación, conversión, plantilla o canonicalización.
- Validación: condición que acepta o rechaza.
- Sink: operación sensible, como consulta, ejecución, apertura de archivo o petición de red.

Ejemplo:

```
query `id` -> URL decode -> conversión omitida -> concatenación SQL
           -> execute() -> filas/error/tiempo
```

Taint y confianza

Los datos no dejan de ser controlables por pasar por una variable interna. La pregunta es si una transformación garantiza la propiedad requerida por el sink. Escapar para HTML no garantiza seguridad para SQL; basename no autoriza un objeto; Base64 no sanea una shell.

Prueba mínima

No empieces extrayendo datos. En un laboratorio, busca primero una diferencia reproducible que solo pueda explicar el parser relevante: error de sintaxis controlado, condición verdadera/falsa o tiempo, con controles negativos. Después identifica contexto, motor y número de columnas antes de ampliar impacto.

Reconstruir la consulta antes del payload

El endpoint `/items?id=7` devuelve un artículo. `id=7-0` devuelve el mismo; `id=7-1` devuelve el artículo 6; `id=7x` responde `400 integer required`.

Observación: algunas expresiones aritméticas parecen evaluarse, pero letras se rechazan. Qué sé realmente: el comportamiento es compatible con SQL numérico, pero también con un evaluador o conversión previa. Hipótesis: H1, la aplicación concatena `id` en SQL; H2, evalúa una expresión y luego parametriza el resultado. Experimento: comparar una pareja booleana válida para el motor sospechado y revisar el error de una construcción que el evaluador local aceptaría pero SQL no, manteniendo un control de ID inexistente. Resultado del escenario: `7 AND 1=1` devuelve el artículo y `7 AND 1=2` no devuelve filas; el log de laboratorio muestra `WHERE id = 7 AND 1=2`. Conclusión: existe un oráculo booleano en contexto numérico. Una comilla no era necesaria y habría probado un contexto distinto.

Contextos que no se parametrizan igual

Necesidad	Forma insegura	Tratamiento seguro
Valor de id	WHERE id = + entrada	marcador y binding entero
Texto de búsqueda	LIKE '% + entrada + %'	marcador y valor construido fuera de SQL
Columna de orden	ORDER BY + entrada	mapa cerrado name -> users.name

El tercer caso no se resuelve enviando el nombre de columna como valor parametrizado: se selecciona una expresión SQL ya definida por el programa.

Mitigación

- Consultas parametrizadas para valores.
- Tipos y allowlists para identificadores o decisiones dinámicas.
- Cuenta de base de datos con privilegio mínimo.
- Errores externos genéricos y logs internos suficientes.
- Pruebas unitarias y de integración sobre la frontera source-sink.

Referencias

- [OWASP SQL Injection Prevention Cheat Sheet](#)
- [Fuente interna SQLi](#)

Anterior: [Linux y shell](#). Siguiente: [Metodología](#).

Encoding, normalización y parsers

Objetivos

Distinguir representación de significado, ordenar transformaciones y explicar por qué dos componentes pueden interpretar los mismos bytes de modo diferente.

Encoding no es cifrado

Base64, percent-encoding, Unicode y escapes JSON representan datos; no prueban confidencialidad. Decodificar recupera otra representación sin secreto. El significado aparece cuando un parser consume el resultado.

Ejemplo conceptual:

```
entrada HTTP:      %2e%2e%2fconfig
decodificación URL: ../config
normalización ruta: /srv/config
operación final:   open('/srv/config')
```

Bloquear la primera cadena y ejecutar la tercera es validación en una fase incorrecta.

Orden de procesamiento

Documenta siempre:

```
bytes -> decodificación de transporte -> parser de formato -> coerción de tipo
      -> normalización/canonicalización -> validación -> parser final -> sink
```

No presupongas el orden. Una aplicación puede validar antes de decodificar, decodificar dos veces o delegar la normalización al sistema de ficheros.

Parser, intérprete y sink

- Un parser convierte una representación en estructura: JSON a objetos, URL a componentes, SQL a árbol sintáctico.
- Un intérprete asigna comportamiento a esa estructura: shell, motor SQL o plantilla.
- Un sink es una operación sensible: ejecutar, consultar, abrir fichero, realizar petición o decidir acceso.

La vulnerabilidad aparece cuando datos controlados alteran estructura o selección de recurso en el sink sin una frontera segura.

Validación textual frente a semántica

"127.0.0.1" no aparece es una comparación textual. la dirección resuelta no pertenece a loopback, privada ni metadatos es una condición semántica. La segunda debe aplicarse al destino final y repetirse tras redirecciones o cambios de resolución cuando corresponda.

Cómo diagnosticar

1. Parte de una entrada válida.
2. Cambia una sola representación.
3. Observa errores de capa: URL, JSON, plantilla, SQL, shell o filesystem.
4. Compara lo enviado con lo registrado y lo ejecutado cuando el laboratorio ofrece logs.
5. Determina cuántas decodificaciones ocurren.
6. Confirma el significado final con una evidencia inocua.

Error frecuente

Una variante codificada que funciona no demuestra por sí sola que "el filtro se saltó". Puede haber cambiado el transporte, el router o la selección de recurso. La conclusión debe señalar qué componente comparó qué forma.

Laboratorio: localizar la decodificación

Un endpoint recibe `file=notes%2Ftoday.txt` y abre `notes/today.txt`. Con `file=..%252Fsecret.txt` responde `blocked token: ..%2F`; con `file=%252e%252e%252fsecret.txt` intenta abrir `../secret.txt`.

Observación: dos entradas doblemente codificadas alcanzan fases distintas. Qué sé realmente: hay al menos dos representaciones y el mensaje de bloqueo muestra una de ellas. Hipótesis: H1, el filtro se ejecuta tras una primera decodificación y antes de una segunda; H2, dos componentes decodifican rutas distintas. Experimento: enviar marcadores donde solo uno de `%25`, `%2e` y `%2f` cambie, y comparar el valor registrado justo antes del `open()` si el laboratorio lo expone. Predicción: H1 mostrará una transformación consistente en todas las variantes; H2 producirá rutas o errores de componentes diferentes. Resultado del escenario: el log muestra `raw -> urldecode -> filtro -> router urldecode -> open`. Conclusión: no "funcionó el encoding"; la propiedad se validó antes de obtener la forma consumida.

Ejercicio de hipótesis rivales

`%2Fadmin` devuelve 404, mientras `/admin` devuelve 403. Diseña una prueba para separar: (a) el proxy no decodifica `%2F`; (b) el router lo trata como parte de un segmento; (c) existe una regla de autorización solo para la ruta canónica. No concluyas bypass hasta identificar qué ruta vio cada capa.

Referencias

- [RFC 3986. percent-encoding y normalización](#)
- [Fuente interna: filtros incompletos](#)

Anterior: [Redes](#). Siguiente: [Linux y shell](#).

HTTP, formularios, cookies y sesiones

Objetivos

Leer una petición y una respuesta, localizar entradas controlables, distinguir transporte de estado y predecir qué parser procesará el cuerpo.

Ciclo petición-respuesta

HTTP intercambia mensajes. El cliente envía método, objetivo, campos y, opcionalmente, cuerpo. El servidor selecciona un recurso, aplica autenticación y autorización, procesa la representación y devuelve estado, campos y cuerpo. HTTP es sin estado por diseño; una sesión es una capa de aplicación que relaciona peticiones mediante un identificador.

```
POST /login?next=/panel HTTP/1.1
Host: laboratorio.local
Content-Type: application/x-www-form-urlencoded
Cookie: session=abc123
```

```
user=ana&password=prueba
```

Entradas distintas: ruta /login, parámetro next, cabecera Host, cookie session y campos del cuerpo. No las procesa necesariamente el mismo componente.

Métodos, cabeceras y estados

- GET solicita una representación y suele transportar parámetros en la query.
- POST entrega datos para que el recurso los procese; no implica por sí solo crear algo.
- PUT, PATCH y DELETE expresan reemplazo, cambio parcial y eliminación, pero la aplicación decide qué autoriza.
- Content-Type declara cómo interpretar el cuerpo; Accept expresa qué respuesta prefiere el cliente.
- Un 2xx indica que HTTP completó la operación, no que la lógica sea segura.
- Un 3xx dirige al cliente; un 401 pide autenticación y un 403 rechaza una identidad ya evaluada. La implementación puede usarlos mal.
- Un 5xx puede revelar un error interno, pero no confirma vulnerabilidad.

Formatos de entrada

Formato	Separación	Parser habitual	Error útil
Query string	?a=1&b=2	Parser de URL/framework	parámetro ausente o duplicado
application/x-www-form-urlencoded	a=1&b=2	Parser de formulario	tipo o campo no válido
application/json	JSON	Deserializador JSON	posición y token inválidos
multipart/form-data	límites y partes	Parser multipart/upload	límite, nombre o fichero inválido

Cambiar solo el cuerpo sin cambiar Content-Type prueba una combinación distinta de la que crees. En Repeater conserva primero una petición válida y modifica una variable cada vez.

Cookies, sesiones y autenticación

La cookie es transporte de datos cliente-servidor. La sesión es el estado asociado, normalmente en servidor. La autenticación responde quién eres; la autorización, si esa identidad puede realizar la acción sobre ese objeto.

Preguntas mínimas:

- ¿El identificador cambia tras iniciar sesión o elevar privilegios?
- ¿Cerrar sesión invalida el estado del servidor o solo borra la cookie?
- ¿La autorización se verifica en cada objeto y acción?
- ¿Secure, HttpOnly y SameSite protegen el transporte esperado?
- ¿El cliente oculta una acción o el servidor realmente la rechaza?

Modelo mental

Bytes HTTP -> servidor/proxy -> parser HTTP -> router -> autenticación
-> parser del cuerpo -> lógica -> autorización -> dependencia -> respuesta

Una diferencia entre proxy y backend puede cambiar el significado. Antes de probar un payload, identifica cuántas capas lo decodifican.

Práctica segura

En una petición de laboratorio, cambia por separado query, cookie, cuerpo y método. Registra qué cambia en estado, longitud, cabeceras y contenido. Una diferencia es un indicio; confirma que depende de tu entrada y no de tiempo, caché o sesión.

Caso guiado: ¿sesión, caché o autorización?

Una petición autenticada devuelve el pedido 1042:

```
GET /api/orders/1042 HTTP/1.1
Host: shop.lab
Cookie: session=A7F1
```

Sin cookie responde 401. Con la cookie de otra cuenta responde 200, pero muestra el pedido 1042 de la primera cuenta.

Observación: la cookie es necesaria, pero dos identidades leen el mismo objeto. Qué sé realmente: hay autenticación; todavía no sé si falta autorización por objeto o si una caché sirve una respuesta anterior. Hipótesis: H1, el backend autoriza solo "usuario autenticado"; H2, una caché ignora la cookie. Experimento discriminatorio: cada cuenta solicita dos pedidos propios distintos añadiendo Cache-Control: no-cache y un parámetro inocuo único. Predicción: si H1 es cierta, la cuenta B seguirá leyendo 1042 y podrá consultar otros identificadores; si H2 es cierta, los marcadores o la cabecera alterarán el patrón. Resultado del escenario: B lee 1042 y recibe 403 al pedir un pedido inexistente, con o sin marcador. Conclusión: el router alcanza lógica de objeto y la sesión se reconoce, pero falta comprobar pertenencia. Siguiente paso: documentar el par identidad-objeto-acción; no intentar "romper la cookie".

Ejercicio de parser HTTP

La aplicación acepta este cuerpo:

```
POST /profile HTTP/1.1
Content-Type: application/json

{"name": "Ana"}
```

Al cambiar solo el cuerpo a name=Ana devuelve 400 con Unexpected token 'a' at position 1. La prueba siguiente no es buscar otro payload: es reconocer que el deserializador JSON rechazó el transporte antes de que name alcanzara la lógica. El control correcto es enviar JSON válido con un valor marcador y observar si cambia el perfil.

Resumen

HTTP mueve representaciones; no aporta por sí solo sesiones ni autorización. Cada ubicación de entrada tiene parser, normalización y frontera de confianza propios.

Referencias

- [RFC 9110: HTTP Semantics](#)
- [Conceptos internos de autenticación y Burp](#)

Siguiente: Redes, DNS y URL. Ejercicios: cuaderno de ejercicios.

Linux, procesos, permisos y shell

Objetivos

Diferenciar usuario, proceso, shell, binario, biblioteca e intérprete; leer permisos; y predecir el efecto de quoting, expansiones, pipes y redirecciones.

Entidades distintas

- Usuario/grupo: identidades usadas por el kernel para comprobar permisos.
- Proceso: instancia en ejecución con PID, credenciales, entorno, directorio y descriptores.
- Binario: archivo ejecutable que contiene código o carga un intérprete.
- Biblioteca: código cargado por procesos; no es un proceso por sí misma.
- Shell: programa que lee, analiza, expande y ejecuta comandos.
- Parser/intérprete: funciones; Bash puede cumplir ambas, pero un programa puede recibir argumentos sin shell intermedia.

Esta diferencia decide si `;`, `$()` o `|` tienen significado. Si una aplicación ejecuta un binario con una lista de argumentos, esos caracteres pueden llegar como datos literales.

Permisos y credenciales

Los bits `rwx` se evalúan para propietario, grupo y otros. Directorios requieren `x` para atravesar y `w` para crear/eliminar entradas según permisos del directorio. SUID cambia el identificador efectivo al ejecutar un binario; sudo aplica una política; capabilities dividen privilegios. No son mecanismos equivalentes.

Pipeline de Bash

Según el manual de Bash, de forma simplificada:

1. tokeniza respetando quoting;
2. analiza comandos y operadores;
3. realiza expansiones;
4. aplica redirecciones;
5. ejecuta y recoge estado.

Expansiones relevantes: variables, sustitución de comandos, aritmética, separación de palabras y globbing. Las comillas simples inhiben expansiones; las dobles conservan algunas, como `$variable` y `$(comando)`.

Operadores

Forma	Función	Condición
<code>a; b</code>	ejecutar en secuencia	lo interpreta una shell
<code>a && b</code>	ejecutar b si a devuelve 0	shell
<code>a b</code>	ejecutar b si a falla	shell
<code>a b</code>	stdout de a a stdin de b	shell crea pipe
<code>>, >>, <</code>	redirigir descriptores	shell
<code>\$(...)</code>	sustituir por salida	expansión habilitada

Argumentos y opciones

La shell produce una lista de argumentos. Después, el programa decide si un argumento que empieza por - es opción. -- suele indicar fin de opciones, pero solo si la herramienta lo implementa. Command injection altera la gramática de shell; argument injection conserva el proceso pero altera su lista de argumentos.

Evidencia

Para comprender un fallo en laboratorio, registra usuario real/efectivo, proceso padre, comando o argv, entorno, directorio y permisos de los archivos implicados. "Tengo una shell" no implica TTY completa, permisos elevados ni canal estable.

Experimento: distinguir shell de argv

Una aplicación ejecuta una comprobación de host. Con host=127.0.0.1; id la respuesta muestra literalmente ping: 127.0.0.1; id: Name or service not known.

Observación: ; llegó a ping, pero no separó comandos. Qué sé realmente: el carácter no fue interpretado como operador por una shell antes de llegar al programa. Hipótesis: H1, ejecución directa `execve("ping", ["ping", "-c", "1", entrada])`; H2, shell con el valor correctamente citado. Experimento: usar una entrada que provoque un error propio de opciones, como -h, y observar argv en un wrapper de laboratorio; después comparar con una variante que contenga expansión `$(printf X)`. Predicción: H1 conserva cada cadena como un único argumento; H2 puede expandir solo si el quoting lo permite. Resultado: el log muestra `argv[3]="127.0.0.1; id"`. Conclusión: no hay command injection demostrada; todavía podría existir option injection según cómo se construya argv.

Construcción por capas

Para POST {"host": "127.0.0.1; printf MARK"}, separa siempre:

1. comillas y escapes que hacen JSON válido;
2. el valor resultante entregado por el deserializador;
3. quoting añadido por la aplicación, si existe;
4. gramática de shell o lista argv final;
5. canal donde debería aparecer MARK.

Un 400 JSON refuta la llegada al shell; un error de ping confirma llegada a ping, no ejecución de un segundo comando.

Referencias

- [GNU Bash Reference Manual](#)
- [Conceptos internos de escalada y TTY](#)

Anterior: [Encoding](#). Siguiendo: [Bases de datos y flujo](#).

Redes, DNS, puertos y URLs

Objetivos

Descomponer una URL, explicar resolución y conexión, reconocer loopback/red privada y evitar confundir puerto con servicio.

URL y autoridad

https://usuario@api.laboratorio.local:8443/v1/items?id=7#detalle
scheme userinfo host:port path query fragment

El fragmento normalmente no se envía al servidor HTTP. userinfo puede confundir validaciones caseras. El origen web es la terna esquema, host y puerto normalizados. El path y la query se interpretan después de seleccionar destino y aplicación.

DNS y conexión

1. El cliente obtiene una dirección para un nombre mediante cachés, archivo hosts o DNS.
2. Selecciona IPv4 o IPv6.
3. Abre transporte al puerto indicado o predeterminado.
4. Para HTTPS negocia TLS y valida identidad.
5. Envía HTTP, cuyo Host puede seleccionar un virtual host.

El nombre usado para resolver, el SNI de TLS y Host suelen coincidir, pero son datos distintos. Esta diferencia explica muchas pruebas de vhosts y algunos fallos de validación.

Direcciones relevantes

- IPv4 loopback: 127.0.0.0/8; 127.0.0.1 es el ejemplo habitual.
- IPv6 loopback: ::1.
- Redes privadas IPv4: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16.
- 0.0.0.0 como dirección de escucha no significa que el cliente deba usarla como destino.

Una cadena diferente puede representar el mismo destino después de resolución o normalización. Las defensas deben validar el destino efectivo, no solo la escritura original.

Puerto no equivale a servicio

Un puerto es un número de extremo de transporte. Un servicio puede escuchar en cualquier puerto. El número sugiere una hipótesis; banner, protocolo y comportamiento aportan evidencia.

Observación	Hipótesis	Confirmación mínima
TCP/8080 abierto	Posible HTTP alternativo	enviar petición HTTP válida
UDP/53 responde	Posible DNS	consulta DNS bien formada
445 abierto	Posible SMB	negociación SMB, no solo nombre de puerto

Parsing diferencial

Una validación puede extraer host con una expresión regular y el cliente HTTP con una biblioteca de URL. Si discrepan sobre @, barras, codificación, IPv6 o redirecciones, la validación no protege el destino real. El diagnóstico compara el valor original, el valor normalizado y la conexión observada.

Caso guiado: nombre, conexión y virtual host

api.lab y admin.lab resuelven a 10.10.20.15. curl http://10.10.20.15/ devuelve una página genérica; curl http://api.lab/ devuelve JSON.

Observación: misma IP, contenido distinto. Qué sé realmente: DNS selecciona la dirección, pero no la aplicación final.

Hipótesis: H1, el servidor elige un virtual host mediante Host; H2, la diferencia procede de una ruta o cookie.

Experimento: enviar a la IP la cabecera Host: api.lab sin cookies y repetir con Host: admin.lab. Predicción: H1 produce respuestas distintas al cambiar solo Host; H2 no. Resultado: la IP con Host: api.lab devuelve el mismo JSON. Conclusión: la selección ocurre después de conectar, en el servidor HTTP. Transferencia: al enumerar un puerto, una respuesta por IP no descarta otros sitios en el mismo socket.

Caso de descarte: un puerto no demuestra protocolo

TCP/8080 acepta conexión, pero al enviar GET / HTTP/1.1 cierra el socket. Esto no confirma "HTTP protegido".

Compara un saludo vacío, captura los primeros bytes del servidor y prueba una petición HTTP completa con Host. Si todas cierran igual y un cliente específico negocia un protocolo binario, se descarta HTTP. El número de puerto solo generó una hipótesis.

Resumen

Una URL es una estructura, DNS es resolución y el puerto es un extremo. Separar estas capas permite comprender vhosts, SSRF y pivoting.

Referencias

- [RFC 3986: URI Generic Syntax](#)
- [RFC 1034: Domain Names](#)

Anterior: [HTTP](#). Siguiente: [Encoding y parsers](#).